



INTRODUCCIÓN A LAS BASES DE DATOS

**Trabajo práctico integrador:
Gestión de restaurante**

Grupo nro: 3

**Integrante(s):
Sarubi Mateo,
Lucio Ramos,
Pérez Natasha.**

**Docente(s):
Javier Vescio / Marcelo Wolf**

10-06-2026

INDICE

Objetivo del Proyecto	3
1) Diseñar un modelo conceptual	4
2) Construir el Diagrama Entidad-Relación (DER)	5
Documentación técnica del DER	6
3) Documentación de los scripts SQL	7
Script 1 - DDL	7
Script 02 — DML	8
4) Validación de datos en consultas SQL	9
Consulta 1	9
Consulta 2	9
Consulta 3	9

INTRODUCCIÓN

Objetivo del Proyecto

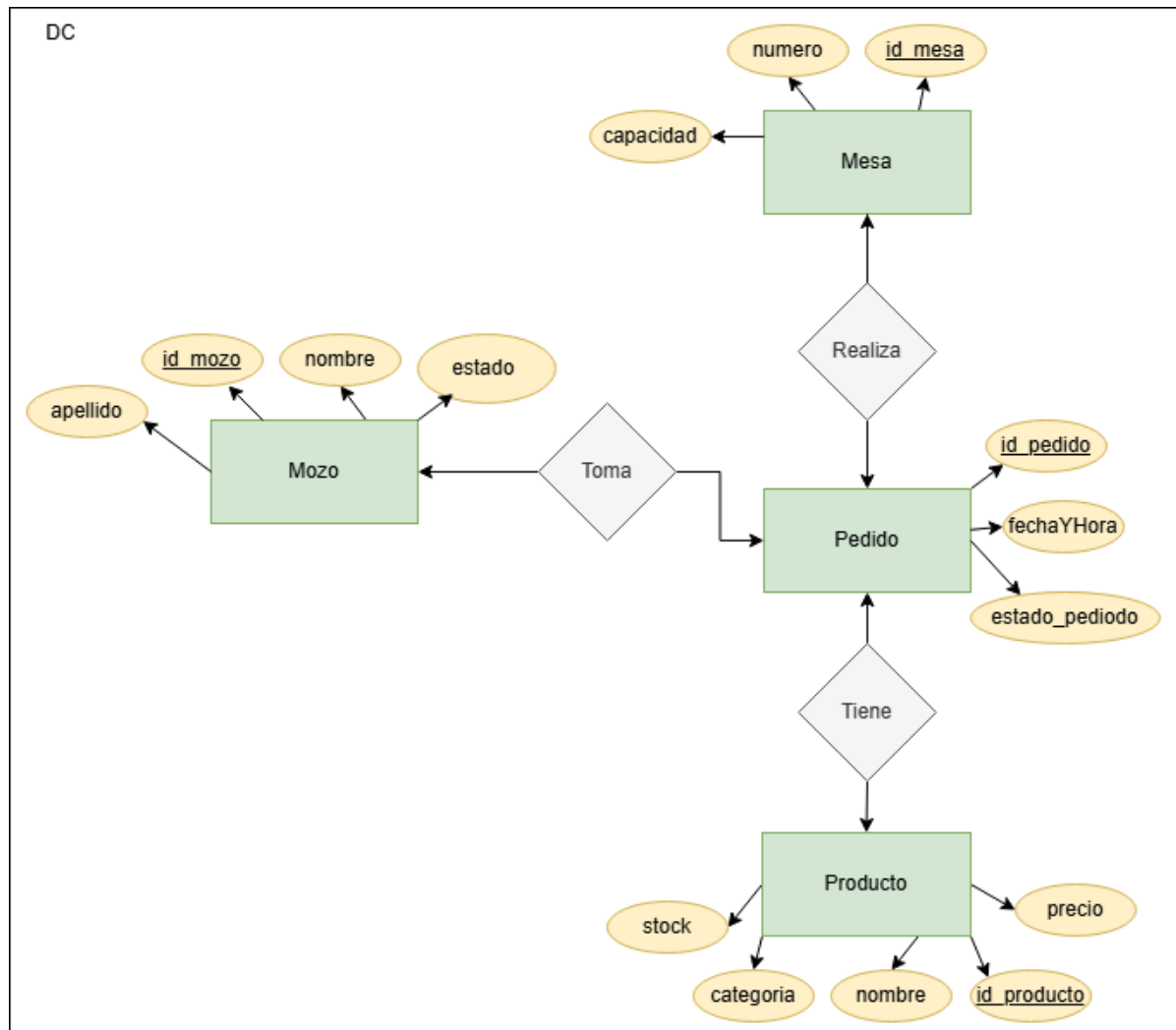
El proyecto Gestión de Restaurante tiene como finalidad desarrollar una base de datos que permita reemplazar el manejo manual de la información por un sistema más organizado y eficiente. Busca centralizar el registro de pedidos, mesas, mozos y productos para facilitar la gestión diaria del restaurante y reducir errores en la operación.

El modelo contempla situaciones reales, como la posibilidad de asociar un mismo pedido a varias mesas cuando se atienden grupos grandes o eventos.

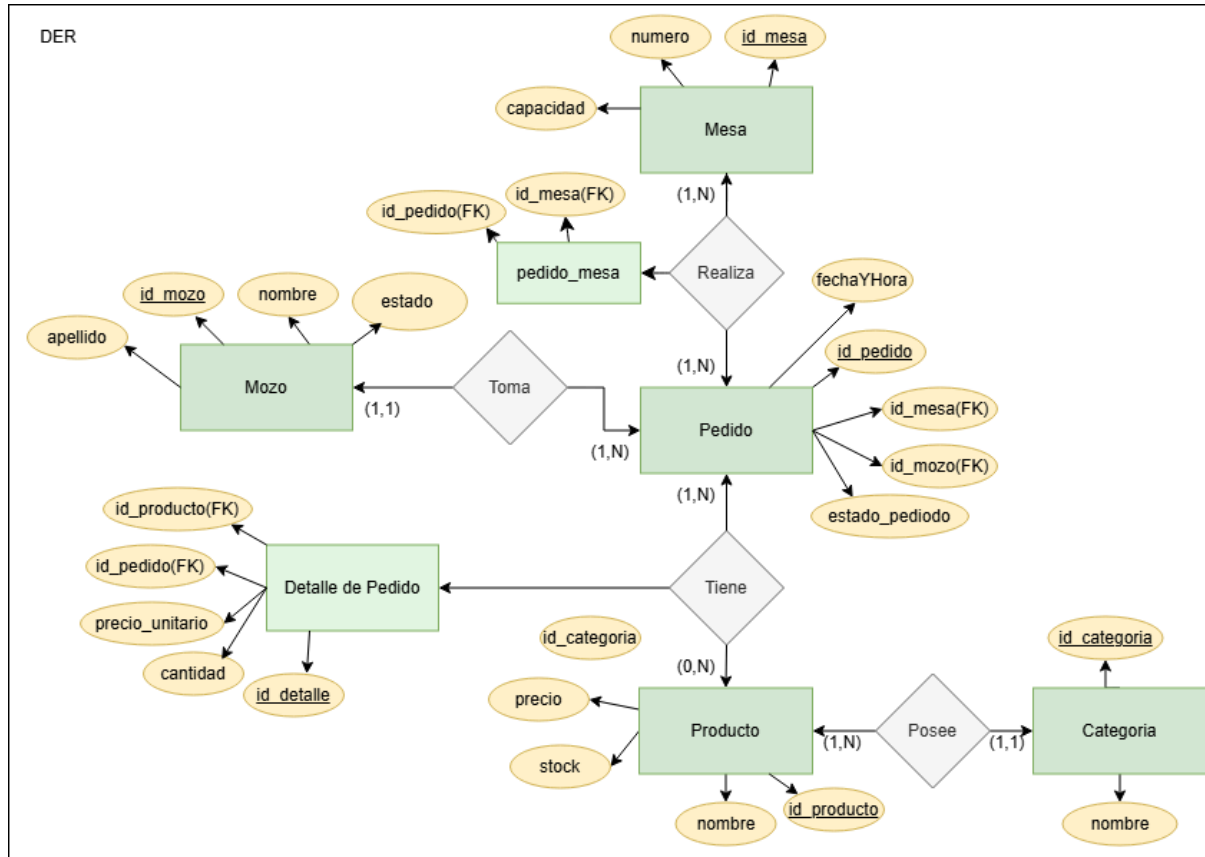
También se busca mantener la información sin redundancias, separando las distintas entidades del sistema, como productos y categorías.

Por último, la base de datos permitirá generar reportes útiles para el restaurante, como facturación por mozo, productos más vendidos, ventas por categoría y control de stock, brindando información para la toma de decisiones.

1) Diseñar un modelo conceptual



2) Construir el Diagrama Entidad-Relación (DER)



Documentación técnica del DER

Se detallan entidades, atributos y claves agregados al DC inicial.

pedido_mesa (MESA - PEDIDO) :

Esta tabla se usa para relacionar mesas y pedidos. Permite que una mesa tenga varios pedidos a lo largo del tiempo y que un mismo pedido pueda estar asociado a varias mesas. Esto resulta útil cuando *se unen mesas para grupos* grandes, cumpleaños o eventos, donde una sola cuenta corresponde a varias mesas.

Contiene los siguientes campos como Clave Primaria Compuesta:

- id_mesa (FK)
- id_pedido (FK)

De esta forma se evita registrar dos veces la misma relación entre una mesa y un pedido.

detalle_pedido (Pedido - Producto) :

Esta tabla relaciona los productos con los pedidos. Un pedido puede contener varios productos y un producto puede aparecer en muchos pedidos diferentes.

categoria

La categoría se modeló como una entidad independiente para evitar repetir datos y mantener la información organizada.

Cada producto pertenece a una categoría, generando una relación 1:N (una categoría puede tener muchos productos).

3) Documentación de los scripts SQL

Script 1 - DDL

Se crea la base de datos restaurante y se define la estructura completa de sus tablas, incluyendo claves primarias, foráneas y tipos de datos.

Tablas creadas

Categoria

Almacena las categorías del menú.

Decisión de estructura: se la separó en una tabla propia para evitar redundancia y permitir agregar o modificar categorías sin tocar la tabla producto. Esta entidad fue incorporada al DER respecto del DC.

id_categoria es un identificador único.

nombre indica el nombre de la categoría.

Producto

Registra los ítems del menú del restaurante.

Decisión de estructura: id_producto es un identificador único.

nombre indica el nombre del producto.

precio almacena el precio base del producto.

stock indica la disponibilidad actual del producto.

id_categoria es una clave foránea que indica la categoría a la que pertenece el producto.

Mozo

Representa al personal de salón que toma pedidos.

Decisión de estructura: id_mozo es un identificador único.

nombre indica el nombre del mozo

apellido indica el apellido del mozo

estado indica el estado laboral del mozo, esto nos permite dar de baja un mozo sin eliminarlo de la base de datos, porque si se borrara se perderían todos los pedidos que tomó.

Mesa

Representa las mesas físicas del salón.

Decisión de estructura: id_mesa es un identificador único.

numero indica el número visible de la mesa en el salón.

capacidad indica la capacidad máxima de clientes en una mesa.

Pedido

Registra cada pedido realizado durante el servicio.

Decisión de estructura: id_pedido es un identificador único.

id_mesa es una clave foránea que indica la mesa donde se realizó el pedido.

id_mozo es una clave foránea que indica el mozo que tomó el pedido.

fechaYHora indica la fecha y hora del pedido.

estado_pedido permite gestionar el estado actual del pedido: pendiente, en_preparacion, entregado o cancelado si el cliente no continúa, sin la necesidad de borrar registros.

Pedido_Mesa

Esta tabla intermedia existe con el fin de formalizar la idea de una relación N a M entre mesas y pedidos para poder utilizar en el sistema del restaurante la opción de “Unir mesas” en un pedido, ya que esto es una práctica muy común en los locales gastronómicos. Esto logra que en el futuro sistema que implemente esta base de datos pueda detallar que hay mesas no disponibles para ocupar por un nuevo pedido ya que se unificó con otra.

Detalle_Pedido

Tabla intermedia que resuelve la relación n:m entre Pedido y Producto, cada fila representa un ítem dentro de un pedido.

Decisión de estructura: id_detalle es un identificador único.

id_pedido es una clave foránea indicando el pedido al que pertenece.

id_producto es una clave foránea indicando el producto solicitado.

cantidad indica las unidades pedidas del producto solicitado.

precio_unitario indica el precio por unidad del momento del pedido.

Script 02 — DML

Carga datos de ejemplo en todas las tablas y demuestra el uso de instrucciones insert, update y delete.

Datos insertados utilizando la instrucción insert fueron:

- 5 categorías para el menú
- 16 productos distribuidos entre las categorías
- 4 mozos activos
- 10 mesas, algunas con la misma capacidad y otras no
- 10 pedidos en distintos estados y fechas
- 37 ítems que componen los pedidos

La instrucción update fue utilizada para:

- actualizar el estado de un pedido, de pendiente a en preparación, indicando así que la cocina tomó el pedido pendiente y comenzó a prepararlo.
- actualizar el estado de un mozo, de activo a inactivo, indicando así la baja del mozo, preservando su historial de pedidos.
- actualizar el precio de un producto ante un error de la carga de datos.

La instrucción delete fue utilizada para demostrar eliminar un ítem de un pedido que fue pedido por error, solo si el pedido está en estado pendiente, por lo que en el primer delete no se concretó la instrucción debido a que el pedido tiene el estado de entregado mientras que en el segundo delete si se concreta la eliminación del ítem del pedido.

4) Validación de datos en consultas SQL

Consulta 1

Esta consulta está hecha pensando en un reporte de facturación hecha por mozo contando la cantidad de pedidos con COUNT y sumando con SUM el precio * cantidad de cada ítem, pudiendo saber cuanto de comisión le correspondería a cada uno. Esto lo hacemos filtrando solamente los mozos activos con un WHERE hacia "mozo.estado" tomando en cuenta los pedidos que tengan como estado "entregado".

Consulta 2

Esta la hicimos pensando en saber cual es el producto más vendido de una categoría (sumando la cantidad de cada detalle vendido), los ingresos generados y cuánto stock queda, así se puede estimar que reponer antes. Evitamos tomar en cuenta con un WHERE los pedidos con estado cancelado en el atributo "pedido.estado_pedido".

Consulta 3

La última consulta la hicimos pensando en mostrar la manera de resolver la función de "unir mesas", mostrando cuál fue la facturación hecha en ese pedido grupal. Usamos Distinct para que no se nos repita el id de una de las mesas unidas, así podíamos contar la cantidad sin errores. La consulta la podemos completar usando los joins para rescatar los datos de Mesa, Detalle pedido y pedido.